91.4
Pass

Rajni Bala

PDF generated at: 18 Feb 2026 18:18:32 UTC

View this report on HackerRank

Score

91.4% • 160.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 46 min 1 sec on 18 Feb 2026 23:00:35 IST

Candidate Information

Email	rajni531095@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	18 Feb 2026 23:00:35 IST
Time taken	46 min 1 sec/ 90 min
Work Experience	5 years
Invited by	Kalyani
Invited on	18 Feb 2026 11:01:12 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	0%

2	Spring Boot Intermediate	66%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	25%	Hard	0%
TDD	0%	Spring	50%
JPA	0%	Spring Data	0%
Transactional	0%	Medium	96%
Pointers	100%	Java	100%
Multithreading	100%	Interfaces	100%
OOP	100%		

Questions

Spring Boot (Advanced) • 10 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status

1	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 33 sec	✗
2	Response Multiple Choice	Spring Boot (Advanced)	0/5	-	49 sec	✗
3	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	0/5	-	2 min 4 sec	✗
4	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	51 sec	✓
5	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	53 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Quiz Competition Coding	Problem Solving (Intermediate)	75/75	-	12 min 21 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	26 min 9 sec	

1. JSON Deserialization

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
    private String name;
    private String surname;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getSurname() {
        return surname;
    }
    public void setSurname(String surname) {
        this.surname = surname;
    }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {

    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
```

```
st) {
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
    "checker":{
        "name": "Checker",
        "surname":"Number2"
    }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn't add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ● **@JsonRootName("checker")**
- ○ @JsonName("checker")
- ○ @JsonTypeName("checker")

@JsonTypeInfo(include = JsonTypeInfo.As.WRAPPER_OBJECT, use = JsonTypeInfo.Id.NAME)
@JsonTypeName("checker")

No comments.

2. Response

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
   @DisplayName("Filter Price Check 3")
   public void FilterPriceCheck3() throws Exception {
      mockMvc.perform(get("/filter/price/150/500"))
            .andDo(print())
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
   }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

[{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]

[{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]

[{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]

[{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]

No comments.

3. Spring Data Transactional

Incorrect

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
    @Id
    private Long id;
    private String name;
    private Integer courseLength;
    private Integer courseLevel;
    @JsonBackReference
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id")
    private Student student;
```

```
    private boolean inProgress;
    //getter & setters
}
```

Student.java

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
    private String surname;
    @JsonManagedReference
    @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
    private List<Lesson> lessons;
    //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods

    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);

        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
```

```
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] It will try to delete the student then the DataIntegrityViolationException will be thrown.

- [x] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".

- [x] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.

- [ ] It will throw a RuntimeException with the message "Student not found".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

4. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- ● **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }** ✓
- ○ None of the above

No comments.

---

5. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope("request") creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope("request").
- [x] @Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

6. Quiz Competition

Correct

Coding Medium Pointers

Question description

A class has students with various talents, each represented by an integer from 1 to *talentsCount*. You need to form teams for a quiz competition, where each team must have at least one member with each talent.

Teams must be formed from consecutive students in the array. For each possible starting position, determine the minimum number of students needed to form a valid team. If it is not possible to form a team with all talents from a particular starting position, return -1 for that position.

Example

talentsCount = 3

talent = [1, 2, 3, 2, 1]

- Starting at position 1: [1, 2, 3] includes all talents, minimum size = 3
- Starting at position 2: [2, 3, 2, 1] is the smallest subarray with all talents, minimum size = 4
- Starting at position 3: [3, 2, 1] includes all talents, minimum size = 3
- Starting at positions 4 and 5: Cannot form a team with all talents, return -1

The result is [3, 4, 3, -1, -1].

Function Description

Complete the function *teamSize* in the editor with the following parameter(s):

int talent[n]: the students' talents

int talentsCount : the number of talents represented numbered from 1 to *talentsCount*

Returns

int[n]: at each index, the minimum size subarray required, or -1 if an appropriate subarray does not exist

Constraints

- $1 \le n, talentsCount \le 10^5$
- $1 \le talent[i] \le talentsCount$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of elements in *talent*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *talent[i]*.

The next line contains the integer *talentCount*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
8       talent[] size n = 8
1       talent = [1, 1, 2, 2, 3, 1, 3, 2]
1
2
2
3
1
3
2
3       talentsCount = 3
```

Sample Output

```
5
4
4
3
4
3
-1
-1
```

Explanation

The shortest subarrays for each position:

Beginning at the first position: [1, 1, 2, 2, 3] (length 5)

Second position: [1, 2, 2, 3] (length 4)

Third position: [2, 2, 3, 1] (length 4)

Fourth position: [2, 3, 1] (length 3)

Fifth position: [3, 1, 3, 2] (length 4)

Sixth position: [1, 3, 2] (length 3)

No further subarrays will have 3 elements.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
5       talent[] size n = 5
1       talent = [1, 1, 1, 1, 1]
1
```

```
1
1
1
1      talentsCount = 1
```

Sample Output

```
1
1
1
1
1
```

Explanation

Since there is only 1 type of talent, any 1 person can form a team.

▼ SAMPLE CASE 2

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
9       talent[] size n = 9
7       talent = [7, 5, 3, 4, 6, 1, 7, 2, 4]
5
3
4
6
1
7
2
4
7       talentsCount  7
```

Sample Output

```
8
7
-1
-1
-1
-1
-1
-1
-1
```

Explanation

There have to be at least *7* elements in any subarray to have one from each talent. The shortest subarrays at each position are:
Beginning at the first position: [7, 5, 3, 4, 6, 1, 7, 2] (length 8)
Second position: [5, 3, 4, 6, 1, 7, 2] (length 7)

No further subarrays will have 7 elements.

Interviewer guidelines

▼ SOLUTION

This can be solved using two-pointer technique. For a particular left index (L), we move the right index (R) forward until we have k distinct elements, then once we get it, we can move L forward. Each element will be traversed at most two times.

Time Complexity: O(n) or O(nLogn) (with set/map).

C++ Solution:

```
int f[100005];

vector<int> teamSize(vector<int> talent, int talentsCount) {
    int k = talentsCount;
    int n = talent.size();
    assert(n>=1&&n<=100000);
    assert(k>=1&&k<=n);
    for(int i=0;i<n;i++)
    {
        assert(talent[i]<=k&&talent[i]>=1);
    }
    int cnt = 0;
    int j = 0;
    vector<int> ans(n);
    for(int i=0;i<n;i++)
    {
        int x = talent[i];
        f[x]++;
        if(f[x]==1)
            cnt++;
        while(cnt==k&&j<=i)
```

```
        {
            ans[j] = i - j + 1;
            int y = talent[j];
            f[y]--;
            if(f[y]==0)
                cnt--;
            j++;
        }
    }
    while(j<n)
        ans[j++] = -1;
    return ans;
}
```

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'teamSize' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY talent
     *  2. INTEGER talentsCount
     */

    public static List<Integer> teamSize(List<Integer> talent, int
talentsCount) {
```

```
    int n= talent.size();
    List<Integer> result = new ArrayList<>();
    int[] freq = new int[talentsCount + 1];
    int unique =0;
    int right = 0;
    for(int left=0; left<n; left++){
        while(right<n && unique< talentsCount) {
            int val = talent.get(right);
            if(freq[val] == 0) unique++;
            freq[val]++;
            right++;
        }
        if(unique==talentsCount){
            result.add(right-left);
        }else {
            result.add(-1);
        }
        int remove = talent.get(left);
        freq[remove]--;
        if(freq[remove]==0) unique--;
            }
            return result;

    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int talentCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> talent = IntStream.range(0, talentCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());
```

```

        int talentsCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> result = Result.teamSize(talent, talentsCount);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.1353 sec	31.6 KB
TestCase 1	Easy	Sample	Success	1	0.135 sec	31.9 KB
TestCase 2	Easy	Sample	Success	1	0.1294 sec	32.1 KB
TestCase 3	Easy	Hidden	Success	3	0.1819 sec	40 KB
TestCase 4	Easy	Hidden	Success	3	0.1687 sec	39.6 KB
TestCase 5	Easy	Hidden	Success	3	0.1489 sec	38.6 KB
TestCase 6	Easy	Hidden	Success	8	0.3255 sec	114 KB

TestCase 7	Easy	Hidden	Success	8	0.3314 sec	115 KB
TestCase 8	Easy	Hidden	Success	8	0.3481 sec	114 KB
TestCase 9	Easy	Hidden	Success	8	0.3557 sec	115 KB
TestCase 10	Easy	Hidden	Success	8	0.286 sec	95.1 KB
TestCase 11	Easy	Hidden	Success	8	0.366 sec	114 KB
TestCase 12	Easy	Hidden	Success	8	0.3562 sec	111 KB
TestCase 13	Easy	Hidden	Success	7	0.3534 sec	82.8 KB

No comments.

7. Java: Safe Cracking

Correct

Coding Medium Java Multithreading Interfaces OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.

2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number
3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- *1* ≤ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ *10*

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
   public void rob(IActions actions) throws InterruptedException, ExecutionException{
      ExecutorService threadPool = Executors.newFixedThreadPool(2);
      String passcodeOfDoor = actions.unlockTheDoor();
      Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
      Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));

      String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
      String safeDepositBoxPin = hackSecretPinTask.get();

      actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
      threadPool.shutdown();
   }
}
```

Candidate's Solution

Language used: **Java 8**

```java
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
```

```
}

import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}

class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{
        String passcodeOfDoor = actions.unlockTheDoor();
        ExecutorService  executor = Executors.newFixedThreadPool(2);
        Future<String> pinFuture = executor.submit(() ->
        actions.hackSecretPin(passcodeOfDoor)
        );
        Future<String> boxNumberFuture = executor.submit(()->
        actions.figureOutSafeDepositBoxNumber(passcodeOfDoor));

        String safetyDepositBoxPin = pinFuture.get();
        String safetyDepositBoxNumber = boxNumberFuture.get();
        executor.shutdown();
        actions.openSafeDepositLock(safetyDepositBoxNumber, safetyDepositBoxPin);


    }
}

public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
```

```
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
     int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

```

```
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(3000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(3000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
```

```
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	3.122 sec	31.8 KB
Testcase 1	Hard	Hidden	Success	74	3.1148 sec	31.6 KB

No comments.